6th ICAIR – 2026
Conference Proceedings of the 6th International Conference on Artificial Intelligence and Robotics (November 3-5, 2026)
Machine Intelligence Research Group (MIRG)
Faculty of Computing and Informatics, University of Lagos

Navigating the Shielding Paradox: Runtime Shared Autonomy for Safe Learning-Based Mobile Robot Navigation

Sunday Ikechukwu[1,2]
[1]Department of Mechatronics Engineering, University of Benin, Nigeria; [2]Central Industrial Liason & Placement Unit (CILPU), University of Lagos, Nigeria
Correspondence: *sundayike.eng@gmail.com*

Abstract

Deploying learning-based navigation policies on mobile robots is challenging because covariate shift can lead to states poorly represented in the training data and increase the risk of navigation failures. Behavior cloning can be improved through expert-labelled recovery data, but policy retraining alone does not provide a formal runtime safety guarantee. This paper presents a runtime shared-autonomy framework that arbitrates control between a learned navigation policy and a Model Predictive Path Integral (MPPI) expert controller using geometric risk criteria and policy–expert disagreement. A recovery-centric data-collection pipeline retains expert-labelled takeover windows, including a 10 control steps (1.0 s of wall-clock time) pre-trigger history, for subsequent policy retraining. The framework is evaluated in ROS 2 and Gazebo across two static indoor simulation environments, with one run per scenario. In the known environment, the final retrained policy completes all eight scenarios, with two collision events in one scenario. In the unseen environment, it completes six of eight scenarios, matching the standalone MPPI controller's completion count, although its total collision count remains similar to that of the initial behavior-cloning policy. Under runtime arbitration, all eight known-environment scenarios and four of eight unseen-environment scenarios reach the goal; the remaining four terminate following expert-planner deadlocks. These results illustrate the limitations of planner-based shielding: intervention can support task completion, but restrictive planning constraints can also prevent progress. Full-run collision counts were not recorded for the monitored system, so its overall collision-avoidance performance remains undetermined.

Keywords: Mobile Robot Navigation, Policy Arbitration, Runtime Shared Autonomy, Safe Imitation Learning, Shielding Paradox

1. Introduction

Learning-based navigation policies can encounter states that are poorly represented in their training data during closed-loop deployment, increasing the risk of navigation failures. Behavior cloning is particularly susceptible to this covariate shift because errors made during execution can lead the policy away from the state distribution represented by expert demonstrations (Ross et al., 2011). Interactive imitation-learning methods address this problem by collecting expert labels for states visited by the learned policy. However, improving a policy through additional training does not by itself prevent collisions during deployment, motivating the use of runtime intervention mechanisms. Reliable low-cost navigation of this kind is of practical interest in sectors such as logistics, agriculture and manufacturing, including in African industry.

Runtime safety methods include control barrier functions, reachability-based approaches, and predictive safety filters, which can enforce constraints under their respective modelling and feasibility assumptions (Ames et al., 2017; Herbert et al., 2017; Wabersich & Zeilinger, 2021). A different practical approach is to transfer control from a learned policy to a classical navigation planner when intervention criteria are met. Such a design introduces a potential limitation: although the planner can recover from some policy failures, its own constraints may prevent it

Cite this article as: Balogun, D. T. & Akinremi, O. O. (2026). Beyond the Face: A Cross-Sectoral Review of Dataset Bias in AI Systems Deployed in African Contexts. In V. Odumuyiwa, & O. Sennaike (Eds.), *Proceedings of the 6th International Conference on Artificial Intelligence and Robotics (MIRG-ICAIR 2026)* (pp. 1–12). Machine Intelligence Research Group, University of Lagos.

© 2026 The Authors. This work is licensed under a Creative Commons Attribution BY-NC-ND 4.0. For more information, see https://creativecommons.org/licenses/by-nc-nd/4.0/

from making progress in narrow or cluttered geometries. We refer to this potential conflict between intervention and task completion as the *shielding paradox*.

This paper presents a runtime shared-autonomy framework that combines a learned mobile-robot navigation policy with the Nav2 Model Predictive Path Integral (MPPI) controller. The long-term aim is for the learned policy to replace the MPPI controller, which serves as the expert and the performance reference. A runtime monitor evaluates obstacle proximity, reversing collision risk, costmap-inflation proximity, and policy–expert disagreement. The arbitrator transfers control to MPPI when a monitored condition is met. During expert takeovers, a recording pipeline retains the preceding 10 control steps (1.0 s of wall-clock time) of observations and subsequent recovery states, pairing them with MPPI commands for offline policy retraining. The pre-trigger buffer is intended to preserve context leading up to intervention; its individual contribution is not evaluated separately.

The main contributions of this work are:

1. A runtime arbitration architecture that transfers control between a learned navigation policy and an MPPI expert according to geometric risk criteria and policy–expert disagreement.
2. A recovery-centric data-collection pipeline that aggregates expert-labelled takeover windows, including a pre-trigger history, for subsequent behavior-cloning retraining.
3. An empirical evaluation of standalone policies and the monitored system in two static simulated environments, documenting both successful navigation and cases in which expert-planner deadlock limits task completion.

The study is limited to simulation and single-run evaluations of each scenario. The results therefore characterize the observed behavior of the implemented framework rather than establish formal collision-avoidance guarantees or statistically generalizable performance.

2 Related Work

2.1 Safe Imitation Learning

Behavior cloning learns a policy from expert state–action demonstrations but can be sensitive to covariate shift when its own errors lead to states that are underrepresented in the training data. Dataset Aggregation (DAgger) addresses this problem by iteratively collecting expert labels for states visited by the learned policy (Ross et al., 2011). Related methods modify how expert guidance is obtained or how safety-critical states are represented during training. HG-DAgger uses human intervention to guide data collection (Kelly et al., 2019), while SafeDAgger uses a learned safety measure to reduce unnecessary expert queries (Zhang & Cho, 2017). DART injects structured noise during expert demonstrations to expose the learner to states arising from imperfect execution (Laskey et al., 2017). SAFE-GIL uses safety-guided disturbances to improve coverage of safety-critical states during imitation learning (Ciftci et al., 2025), while SafeIL incorporates explicit safety constraints into imitation learning (Lee et al., 2026).

Our approach focuses on data retained during runtime planner takeovers. Rather than retaining all deployment states, the recording pipeline stores selected takeover windows and includes a short pre-trigger history labelled with the MPPI command computed at each state. This design is intended to preserve context leading to intervention while concentrating the retained dataset on recovery-related states. The study does not compare this data-selection strategy against full-state DAgger or isolate the effect of the pre-trigger buffer through an ablation.

2.2 Runtime Safety and Autonomous Arbitration

Control barrier function methods formulate safety constraints that can be incorporated into control optimization, with guarantees dependent on the system model and assumptions used in the formulation (Ames et al., 2017). Reachability-based methods such as FaSTrack provide a framework for accounting for tracking error and obtaining safety guarantees under specified assumptions (Herbert et al., 2017). Predictive safety filters assess proposed control actions against constraints using a predictive model and uncertainty estimates (Wabersich & Zeilinger, 2021). These approaches provide principled ways to constrain control, but their guarantees depend on their respective assumptions and are not automatically provided by a conventional navigation planner.

The Nav2 MPPI controller (Macenski et al., 2020; Williams et al., 2017) offers a sampling-based local-planning approach that evaluates candidate trajectories using the navigation costmap and reference path. In this work, it serves as the expert controller and as the fallback during monitored interventions. Unlike a formal safety filter, the implemented arbitration mechanism uses hand-set geometric thresholds and command-disagreement criteria; it is therefore evaluated empirically and does not provide a formal guarantee of collision avoidance. Conservative fallbacks can also stall a robot. Switching to a conservative controller follows the Simplex architecture (Sha, 2001), and stalling under conservative behavior is a

Author Proof

known problem in navigation, the "freezing robot" problem (Trautman & Krause, 2010).

Shared-autonomy research has also investigated how autonomous systems combine their decisions with human input. For example, Broad et al. (2017) study shared control for human–machine systems with unknown dynamics, while Reddy et al. (2018) use deep reinforcement learning to combine human input with autonomous assistance. These works address a different control-allocation setting from the one considered here. Our framework transfers control between two autonomous controllers, using geometric risk and policy–expert disagreement rather than inferred human intent. The central question is whether this planner-based intervention can help a learned navigation policy while also exposing cases in which the expert planner itself becomes stuck.

3 Problem Formulation

3.1 Navigation Setting

We consider a mobile robot navigating a two-dimensional, obstacle-cluttered environment in discrete time. Let $s_t \in \mathcal{S} \subseteq \mathbb{R}^{n_s}$ denote the observation (local environment, current velocities and a goal term) and let the commanded action be the linear and angular velocity, bounded by the platform limits:

$$u_t = [v_t, \omega_t]^\top \in \mathcal{U} \quad (1)$$

A global planner provides a reference path to the global goal. At each step the local controller extracts a lookahead sub-goal on that path, given by its distance and relative bearing in the robot frame. The objective is to reach the goal without collisions.

3.2 Arbitration

A learned policy $\pi_\theta: \mathcal{S} \to \mathcal{U}\ produces\ u_t^\pi = \pi_\theta(s_t)$. Because it may meet states that are poorly represented in its training data, a model-based expert, the MPPI controller, provides u_t^{MPPI}. The expert additionally uses the local costmap and global plan, which are not part of s_t. A runtime arbitrator selects one of the two commands:

$$u_t = (1 - \alpha_t)\, u_t^\pi + \alpha_t\, u_t^{MPPI}, \quad \alpha_t = S_t \in \{0,1\}, \quad (2)$$

where $S_t = 1$ is the monitor status indicating that intervention is required. The monitor's triggers (obstacle proximity, reversing collision risk, costmap-inflation proximity and policy–expert disagreement) and the hysteresis rules are specified in Section 4.3. The monitor applies a geometric clearance criterion. It does not provide a formal guarantee of collision avoidance, and its thresholds are set by design, not by optimization. Its purpose is to respond to likely policy failures and to obtain expert demonstrations. Performance is therefore assessed empirically, through task completion, collision events and intervention outcomes.

3.3 Learning Objective

Let

$$\mathcal{L}(u, u^*) = (v - v^*)^2 + \lambda_w(\omega - \omega^*)^2, \quad \lambda_w = 3.0, \quad (3)$$

be the weighted squared error between a normalized command $u = [v, \omega]^\top$ and an expert target u^*. Behavior cloning trains the policy on a filtered dataset $\mathcal{D}_{\text{initial}}$ of expert state–action pairs:

$$\min_\theta \mathbb{E}_{(s,u^*)\sim\mathcal{D}_{initial}}[\mathcal{L}(\pi_\theta(s), u^*)] \quad (4)$$

A fixed dataset may not cover the states the policy itself visits. When the expert takes control, the pipeline therefore records recovery chunks $\mathcal{D}_{rec,k}$: the states visited during expert control and in the 10 control steps (1.0 s of wall-clock time) before takeover, each paired with the MPPI command computed at that step. After filtering (Section 4.5), these are aggregated with the existing dataset,

$$\mathcal{D}_{k+1} = \mathcal{D}_k \cup \mathcal{D}_{rec,k} \quad (5)$$

and the policy is retrained on $\mathcal{D}_{k+1}$with the same weighted loss. The final policy uses two rounds of recovery data, without oversampling.

4 Methodology

4.1 System Overview

The framework combines a learned navigation policy with the Nav2 MPPI controller, which serves as the expert and as the performance reference the policy is intended to replace. The policy provides the primary commands. A runtime monitor transfers control to the expert on obstacle proximity, reversing collision risk, costmap-inflation proximity, or policy–expert disagreement (Section 4.3). At each step the policy maps the observation s_t to a command u_t^π, while MPPI computes u_t^{MPPI} from the local costmap and global plan; the monitor outputs the binary status S_t, and the arbitrator selects the command according to Eq. (2), with hysteresis to prevent rapid switching. While the expert is in control, a recorder saves recovery data labelled with the MPPI command, including the preceding 10 control steps (1.0 s of wall-clock time) of states; these data are aggregated and used to retrain the policy offline (Sections 4.4 and 4.5). The framework does not introduce a new training algorithm. Instead, it integrates runtime arbitration and recovery-centric data collection into a policy-improvement pipeline. We evaluate in simulation both the retrained policy running alone and the monitored system as a whole. (Figure 1)

Author Proof

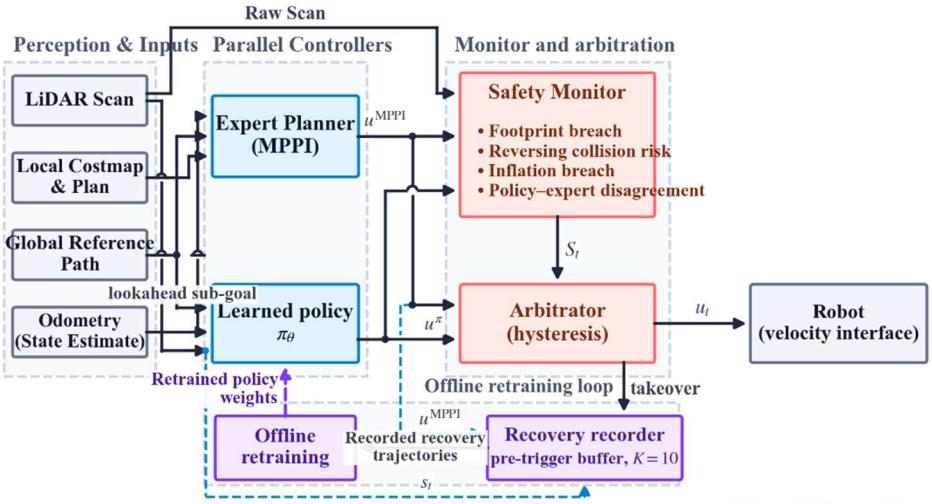


Figure 1. Overview of the monitored policy–expert loop and the offline recovery-data pipeline. ***Source:*** *Authors' creation (2026).*

4.2 System Components

The learned policy runs at 10 Hz through ONNX Runtime. The Nav2 MPPI controller, a sampling-based planner (2,000 sampled trajectories at 20 Hz), runs continuously on the local costmap and global plan as a hot standby. Its command u_t^{MPPI} is therefore available at every step, including while the policy is in control, which lets the monitor compare the two commands at all times and lets the arbitrator hand over without delay. The monitor reads the raw LiDAR scan, the goal status and both commands. The arbitrator is the only publisher to the robot's velocity interface, and the recorder logs observations and MPPI commands (Section 4.4). In this study MPPI therefore runs throughout; the computational saving that motivates replacing it is not measured here.

4.3 Runtime Arbitration

The monitor sets $S_t = 1$ when any condition in Table 1 holds. The conditions are checked in the order listed, and the first to hold is logged as the cause of the takeover; all conditions have the same effect on arbitration. The two proximity levels follow platform values: 0.22 m is the TurtleBot3 Waffle footprint radius and 0.35 m is the Nav2 local-costmap inflation radius. A reading below 0.22 m also satisfies the 0.35 m condition, so the footprint level differs only in how a takeover is logged; it was the logged cause of 5 takeovers in the first recovery dataset and 2 in the second. Disagreement is logged as the cause only when no earlier condition holds. The thresholds were set by hand and were not tuned or tested for sensitivity.

To prevent chattering, control stays with the expert after S_t returns to 0 until a recovery timer $T_{rec} = 3.0$ s has elapsed and the expert's command shows that the robot has settled ($|\omega_t^{MPPI}| < 0.15$ rad/s and $v_t^{MPPI} > 0.05$ m/s). A takeover also ends if the goal is reached or cancelled. The arbitrator's timers use the system clock, not simulation time (Section 4.6).

Table 1. Monitor conditions. $S_t = 1$ if any condition holds.

Condition (checked in this order)	Rule	Basis of threshold
1. Footprint breach	$d_{min} < 0.22$ m	Robot footprint radius
2. Reversing collision risk	$v^{\pi} < -0.01$ m/s and $d_{rear} < 0.25$ m	Set by hand
3. Inflation breach	$d_{min} < 0.35$ m	Nav2 local-costmap inflation radius
4a. Angular disagreement	$\|\omega^{MPPI} - \omega^{\pi}\| > 0.8$ rad/s	Set by hand
4b. Linear disagreement	$\|v^{MPPI} - v^{\pi}\| > 0.15$ m/s	Set by hand
4c. Turn-sign mismatch	sign ωMPPI ≠ sign ωπ, with |ωMPPI| and |ωπ| both > 0.2 rad/s	Set by hand

Source: Authors' creation (2026)

Author Proof

If the expert's commanded linear and angular speeds both stay below 0.015 (m/s and rad/s) for Tdeadlock = 4.0 s, the arbitrator passes control to the learned policy for a 1.5 s shake-up. The policy acts on the raw scan and is not bound by the costmap inflation boundary. If the expert's command remains below the stall threshold after the shake-up duration, the arbitrator cancels the active Nav2 goal, publishes a zero-velocity stop command, and the mission is recorded as terminated. If motion successfully resumes, control either returns directly to the learned policy (if safety risk has cleared) or reverts to the expert controller until the safety hysteresis conditions are satisfied.

4.4 Recovery-Centric Data Collection

Standard DAgger labels every state the policy visits, which fills the dataset with nominal frames, such as straight corridors, where the policy already performs well. We instead record only takeover windows, which concentrates the data on failure regions: reversing, absent from the nominal demonstrations, makes up 14.53% of the second recovery dataset (Section 5.4). We did not compare this choice against full-state labelling.

Recording only the expert's recovery would leave out the states that led to the takeover. The recorder therefore keeps a first-in-first-out buffer of the last K = 10 observation–command pairs at 10 Hz (the pre-trigger buffer). When the arbitrator hands control to the expert, the buffer is flushed into the recovery chunk, and recording continues while the expert is in control. Every recorded observation, including the buffered ones, is labelled with the MPPI command computed at that step, which is available because MPPI runs continuously:

$$\tau = \{(s_{t-K}, u_{t-K}^{MPPI}), \dots, (s_{t+T}, u_{t+T}^{MPPI})\} \quad (6)$$

where T is the length of the expert intervention. The buffered frames are states the policy itself visited, paired with the corrective command the expert would have given there. The separate effect of the buffer on performance was not tested.

4.5 Policy Improvement Pipeline

The policy is improved over rounds. First, MPPI drives the robot through navigation tasks in Environment A, and the successful runs form $D_{initial}$, on which an initial policy π_0 is trained by behavior cloning (Eq. 4). In round k, policy π_k drives while the monitor and arbitrator run, and the recorder collects recovery chunks (Section 4.4). The recorded data are filtered before training. Frames are removed if the commanded speed exceeds 0.05 m/s while odometry speed is below 0.01 m/s over a 10-step window (stuck), or if both velocities are below 0.005 while the goal is farther than 0.15 m (idle). Recovery chunks are dropped if they are shorter than 5 steps, if the expert's mean speed is below 0.05 m/s, or if speed disagreement triggered them and more than 70% of their steps are reversing. Because chunks with a stalled expert are dropped, takeovers in which MPPI stalls add no training data. The filtered chunks are aggregated (Eq. 5), and π_{k+1} is retrained from scratch with the loss in Eq. 3. The neural network is trained using the Adam optimizer with a learning rate of 1e-3, a batch size of 256, and a maximum of 200 epochs with an early stopping patience of 20, utilizing an 80/20 train-validation split. We ran two rounds: π_0 drove the first and π_1 the second. The final policy π_2 is trained on $D_{initial}$ and both recovery sets. Dataset sizes in Section 5.4 are after filtering.

4.6 Implementation Details

The system is built on ROS 2 Humble. Policy inference, the monitor and the arbitrator run at 10 Hz. The observation has 40 dimensions: 36 LiDAR beams (the 360° scan downsampled and clipped to 3.5 m), the current linear and angular velocity (2), and the local goal as distance and bearing (2). The goal is a lookahead sub-goal taken 1.5 m along the global path, which prevents the policy from shortcutting the route. If the distance to the goal is below 0.15 m, the goal angle is set to 0 to avoid a singularity.

The policy is a multilayer perceptron with hidden sizes [256, 128, 64], ReLU activations, dropout 0.1 and a linear two-dimensional output. Observations and actions are standardized using z-scores computed from the training dataset statistics, and the network's outputs are denormalized back into physical units before being clipped to $v \in [-0.26, 0.26]$ m/s and $\omega \in [-1.9, 1.9]$ rad/s.

The expert MPPI planner evaluates 2,000 trajectories per cycle using a DiffDrive motion model, projecting 56 time steps at a 0.05 s resolution (a 2.8 s horizon). The planner enforces maximum kinematic limits identical to the policy clipping limits, utilizing a costmap CostCritic weight of 3.81 and a rigid 1000000.0 collision penalty.

Simulation and inference ran on an Intel Core i5-6200U CPU (2 cores) with 8 GB RAM. With Gazebo Classic, Nav2, MPPI, and the monitoring nodes sharing the CPU, the real-time factor was 0.25–0.35. The policy inference, monitor, and recorder nodes all operate on the system wall-clock alongside the arbitrator. Because the arbitrator's timers (T_{rec}, $T_{deadlock}$ and the shake-up) and the 10 Hz recording loop use the system clock, each corresponds to about 0.25–0.35 times as much simulated time. Consequently, the 3.0 s cooldown timer yields approximately 0.8–1.0 s of simulated time, and the K = 10 pre-trigger buffer spans about 0.3 s of simulated robot motion. The metrics logger and all durations reported in Section 6 use simulation time.

Author Proof

5 Experimental Setup

5.1 Platform and Environments

Experiments use a simulated TurtleBot3 Waffle (ROBOTIS) in Gazebo Classic (Koenig & Howard, 2004) with ROS 2 Humble and Nav2 (Macenski et al., 2020); policies run through ONNX Runtime (ONNX Runtime developers, 2021) on the CPU. Gazebo Classic was used because it is the simulator supported by the TurtleBot3 packages for ROS 2 Humble and it ran the full stack on the available CPU; it reached end of life in January 2025 (Section 7.3). Hardware and simulation speed are given in Section 4.6.

Two indoor environments are used, both with static obstacles only (Figure 2). **Environment A (known)** is an office layout with straight corridors, L-shaped corners and sparse obstacles; all demonstrations and recovery data were collected there. **Environment B (unknown)** has interconnected corridors, blind corners, offset doorways and cluttered furniture, and none of its data was used in training. Each environment has eight evaluation scenarios (A1–A8, B1–B8) with fixed start and goal poses, listed in the released configuration [repository].

5.2 Methods and Experiments

Four configurations are compared: the behavior cloning baseline π_0, trained on $D_{initial}$ only; the retrained policies π_1 and π_2, trained with recovery data from one and two rounds (recovery-centric behavior cloning, RC-BC), where π_2 is the final policy; MPPI, the expert and performance reference; and the monitored system, which is π_2 under the monitor and arbitrator of Section 4.3 with MPPI as the expert. Policies run alone, without the monitor, in Experiments A and B. Experiment A compares MPPI, π_0, π_1 and π_2 in Environment A. Experiment B compares π_0 and π_2 in in Environment B, with the MPPI expert reference. Experiment C runs the monitored system in both environments.

5.3 Metrics and Procedure

One logger scores all runs. Each run records the outcome, navigation time in simulation seconds, path efficiency (actual over planned path length, averaged over completed runs only) and collision events. A collision event is a debounced occurrence in which the minimum LiDAR range drops below 0.24 m (the footprint radius plus a 2 cm margin), counted once per continuous contact period. Monitored runs additionally record the intervention count (a controller transition from the learned policy to the expert triggered by the monitor), successful recoveries (a takeover followed by at least 2.0 s of collision-free policy control), expert control time, the share of time and distance driven by the policy, and the obstacle distance at the moment of takeover.

A run ends when the goal is reached (0.25 m tolerance); when the arbitrator cancels the goal (Section 4.3, monitored runs only); when displacement stays below 0.05 m for 50.0 s; or at an absolute timeout of 300.0 s. A run that ends without reaching the goal is recorded as aborted (standalone) or terminated (monitored). Each scenario was run once (n = 1) because of simulation cost. MPPI is a sampling-based planner, so outcomes may differ between runs (Section 7.3).

5.4 Datasets

Table 2 summarizes the datasets; sizes are after filtering (Section 4.5). Recovery data were collected in Environment A only, while π_0 and then π_1 drove under the monitor. Policy π_2 is trained on all three datasets (55,116 steps). Scenarios A1–A8 have fixed poses and were defined after π_2 was trained. They share the map and behavior types with the training data, so Experiment A measures performance in the known environment, not generalization. Demonstrations cover several scenario types (short and long routes, U-turns, goals between obstacles) from up to 15 start locations per type, with one to two runs per start.

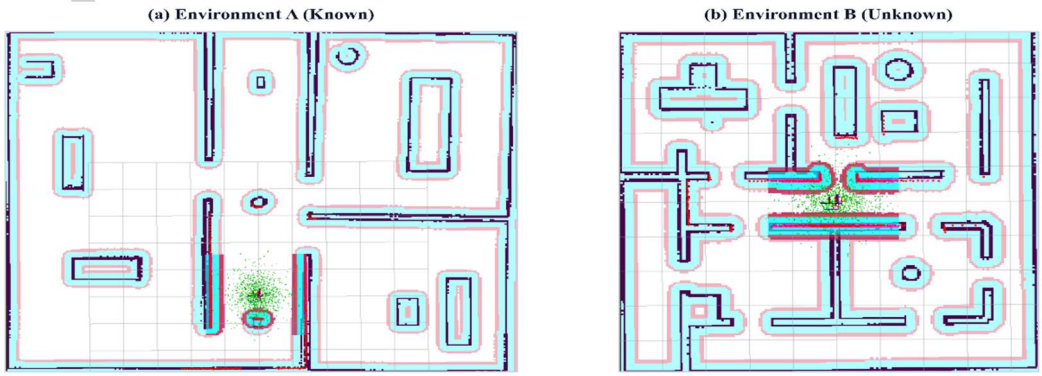


Figure 2. Simulation environments: (a) Environment A (known); (b) Environment B (unknown). **Source:** Authors' creation (2026).

Author Proof

Table 2. Datasets used for training.

Dataset	Collected with	Episodes / chunks	Steps	$\|\omega\| > 0.9$ rad/s	Reversing
$D_{initial}$	MPPI	133 **episodes**	22,343	2.38%	0%
$D_{rec,1}$	π_0 under monitor	523 **chunks**	10,724	20.84%	10.27%
$D_{rec,2}$	π_1 under monitor	566 **chunks**	22,049	28.65%	14.53%

Source: Authors' creation (2026)

Policy π_2 was chosen between variants trained with and without 3× oversampling of the recovery data, using validation error (angular-velocity MAE 0.208 against 0.210 rad/s; linear-velocity MAE 0.028 against 0.030 m/s) and informal observation of closed-loop behavior in simulation. The 80/20 validation split was made by episode for the initial data and by chunk for the recovery data.

6 Results

All results are single runs ($n = 1$) utilizing deterministic start and goal poses. Outcomes and navigation metrics for standalone policies are detailed in Tables 3, while the monitored runs of Experiment C are detailed in Table 4.

6.1 Experiment A: Policy Navigation Performance in Known Environment

Experiment A evaluates the comparative navigation performance and learning progression of the policies in the training environment (Environment A) across scenarios A1 through A8 (Table 3).

The standalone Behavior Cloning (BC) baseline (π_0) exhibits severe performance degradation, failing to complete seven of the eight evaluation scenarios. In these aborted runs, the BC policy frequently collided and subsequently became stuck, leading to timeout-based terminations characterized by unstable control commands. The only scenario successfully completed by the BC policy is A5; however, this success is accompanied by 9 collision events, demonstrating a fundamental lack of safety awareness.

Iterative policy retraining demonstrates substantial progress. The Final RC-BC policy (π_2) successfully completes all eight scenarios, achieving a 100% success rate matching the MPPI expert. The navigation durations show substantial improvement over earlier iterations, achieving efficient trajectories close to the MPPI Expert reference (e.g., A1 in 23.2 s against 20.5 s). Path efficiency remains near-optimal, ranging from 0.97 to 1.09. It records zero collisions in all scenarios except A3, where it encounters 2 collisions but completes the run successfully. Because these scenarios share the map with the training data (Section 5.4), this shows that the retrained policy approaches the expert in the known environment. It does not show generalization, and it cannot separate the effect of recovery data from that of simply having more training data (55,116 against 22,343 steps).

6.2 Experiment B: Policy Generalization under Distribution Shift

Experiment B evaluates the generalization capability of the standalone policies under zero-shot transfer to the unseen Environment B, which features narrower passages and occluded blind corners. The MPPI planner is also evaluated standalone strictly to establish the baseline difficulty of these novel geometries The results are summarized in Table 3.

Table 3. Navigation performance and generalization benchmark across known and unseen environments

Method	Known Environment (Exp. A)			Unseen Environment (Exp. B)		
	Success Rate	Path Eff. (Mean ± SD)	Collisions (Total)	Success Rate	Path Eff. (Mean ± SD)	Collisions (Total)
Behavior Cloning (BC)	12.5% (1/8)	1.41 ± 0.87	26	12.5% (1/8)	0.83 ± 0.39	23
RC-BC (v1)	87.5% (7/8)	1.41 ± 0.48	13	—*	—*	—*
Final RC-BC	100.0% (8/8)	1.02 ± 0.04	2	75.0% (6/8)	1.03 ± 0.37	22
MPPI Expert (Reference)	100.0% (8/8)	0.99 ± 0.03	0	75.0% (6/8)	0.90 ± 0.26	6

Note: Path efficiency is computed as actual path length over planned path length (optimal = 1.00). Collisions denote physical obstacle contact (< 0.24 m). *RC-BC v1 was evaluated on in-distribution scenarios to validate iterative recovery learning prior to unseen generalization testing. **Source:** Authors' creation (2026)

Author Proof

Table 3. Navigation performance and generalization benchmark across known and unseen environments

Method	Known Environment (Exp. A)			Unseen Environment (Exp. B)		
	Success Rate	Path Eff. (Mean ± SD)	Collisions (Total)	Success Rate	Path Eff. (Mean ± SD)	Collisions (Total)
Behavior Cloning (BC)	12.5% (1/8)	1.41 ± 0.87	26	12.5% (1/8)	0.83 ± 0.39	23
RC-BC (v1)	87.5% (7/8)	1.41 ± 0.48	13	—*	—*	—*
Final RC-BC	100.0% (8/8)	1.02 ± 0.04	2	75.0% (6/8)	1.03 ± 0.37	22
MPPI Expert (Reference)	100.0% (8/8)	0.99 ± 0.03	0	75.0% (6/8)	0.90 ± 0.26	6

Note: Path efficiency is computed as actual path length over planned path length (optimal = 1.00). Collisions denote physical obstacle contact (< 0.24 m). *RC-BC v1 was evaluated on in-distribution scenarios to validate iterative recovery learning prior to unseen generalization testing. **Source:** Authors' creation (2026)

The BC baseline (π_0) exhibits severe performance degradation when subjected to structural distribution shift, failing to complete seven of the eight scenarios. In contrast, the MPPI expert completes six of the eight scenarios but struggles in the most constrained geometries, aborting in B6 and B8 due to deadlocks and multiple collisions (5 collisions in B8). This establishes the performance upper-bound and highlights the severe difficulty of the unseen environment.

The Final RC-BC policy (π_2) demonstrates strong resilience, successfully completing six of the eight unseen scenarios, matching the completion rate of the classical expert. While the total number of collision events for RC-BC across all runs (22) is similar to the BC baseline (23), the outcomes are fundamentally different. The BC baseline collides and terminally aborts; the RC-BC policy utilizes its training on failure-recovery trajectories to survive these boundary breaches, stabilize, and successfully reach the goal. Despite this resilient generalization, the RC-BC policy fails in two highly complex scenarios: B4 (blind corner east turn) and B5 (room-to-room transition), aborting after 100.4 seconds and 25.1 seconds, respectively. These failures correspond to geometric constraints that lay outside the training distribution, directly motivating the need for runtime arbitration.

6.3 Experiment C: Runtime Shared Autonomy and Safety Interventions

Experiment C evaluates the runtime shared-autonomy framework, which wraps the Final RC-BC policy with the MPPI expert planner under safety monitor arbitration (Table 4).

In Environment A, the framework achieves stable shared autonomy, successfully completing all scenarios. The system retains high learned-policy autonomy, with the policy driving the robot for the majority of the mission time (ranging from 32.98% to 98.61%) and distance (ranging from 53.85% to 98.62%). In the unseen Environment B, the safety monitor dynamically adapts to the increased complexity. In scenarios B1 through B4, the robot successfully completes the navigation tasks. However, due to the environmental distribution shift, the safety arbitrator triggers more frequent expert interventions (reaching 15 in B1).

For scenarios B5 through B8, the runs were terminated before reaching the goal. These episodes aborted under the expert-stuck recovery logic. Specifically, the robot entered narrow, highly cluttered passages that triggered safety interventions. Once control hand-off occurred, the MPPI expert attempted recovery maneuvers but eventually became stuck due to narrow geometric clearances, exceeding the 4.0-second deadlock threshold and triggering a timeout cancellation.

Crucially, as demonstrated in Experiment B, the MPPI planner also fails and deadlocks in these exact same scenarios (B6 and B8) when running completely standalone. This empirically isolates the cause of the system failure: the learned policy safely hands over control, but the rigid costmap constraints of the classical planner trap it in local minima, confirming the shielding paradox. Because the runtime metrics logger was specifically configured to evaluate intervention dynamics and autonomy retention, cumulative collision and near-miss counts were not recorded for the fully shielded runs in Experiment C. Full-run collision and near-miss counts were not recorded for Experiment C. Although the recorded obstacle distances at takeover indicate that the monitor intervened before the defined collision threshold at those takeover moments, they do not establish that collisions were prevented throughout the monitored runs. The absence of full-run collision counts limits the conclusions that can be drawn

Author Proof

about the framework's overall collision-avoidance performance.

7. Discussion

7.1 Overcoming Action Ambiguity

The failures of the standalone Behavior Cloning (BC) policy align with the known limitations of supervised imitation learning. At geometrically ambiguous decision points, averaging opposing expert demonstrations yields near-zero angular velocities, driving the robot into obstacles. We did not test this explanation. Interactive recovery aggregation directly addresses this by concentrating the dataset on failure boundaries. By logging only expert takeovers and prepending the pre-trigger buffer, the RC-BC policy captures the critical corrective extremities such as reversing (14.53%) and extreme steering (28.65%) than the nominal demonstrations (0% and 2.38%), providing the necessary context to survive and stabilize from out-of-distribution drift. The study cannot, however, separate this from the effect of additional data, or isolate the pre-trigger buffer.

7.2 The Shielding Paradox

The unseen environment exposes a fundamental trade-off between task completion and formal safety. While the unconstrained RC-BC policy successfully completed complex scenarios like B6 and B8, it did so at the cost of multiple physical collisions. The shielded framework prevented these boundary breaches but ultimately aborted the missions.

This disparity stems from the interaction between the safety monitor and the classical expert. In constrained geometries, the learned policy triggers disagreement warnings, transferring control to the expert. The expert, bound by rigid costmap inflation limits, becomes trapped in local minima, a vulnerability confirmed by the standalone MPPI planner's identical deadlocks in B6 and B8. As established in literature regarding conservative safety filters, strict mathematical constraints often limit task feasibility in unstructured spaces. Furthermore, the 4.0-second deadlock timeout is a strict heuristic; a longer stall allowance might permit the planner to find an escape path, representing an area for future temporal tuning.

7.3 Experimental Limitations

The evaluation has several limits. All tests are in simulation with static obstacles only, in Gazebo Classic, which reached end of life in January 2025. All runs used a real-time factor of 0.25 to 0.35 with wall-clock timers, so behavior at real-time speed is untested. Each scenario was run once and MPPI is sampling-based, so outcomes may vary; the statistics in Tables 4 and 5 are across scenarios, not repeated trials. The retrained policy used about 2.5 times as much training data as the baseline (55,116 against 22,343 steps), and the contributions of the pre-trigger buffer, the disagreement triggers and recovery-focused data selection were not isolated by ablation or compared with DAgger. Thresholds were set by hand and not tested for sensitivity. Collisions were not recorded for the monitored runs. The computational saving that motivates replacing MPPI was not measured.

8. Conclusion

This paper presented a runtime shared-autonomy framework that mitigates covariate shift by arbitrating control between a learned neural policy and a classical model-based planner. Recovery-centric data aggregation successfully enabled the policy to survive boundary breaches, matching the expert's completion rate in both known and unseen environments.

However, runtime evaluation exposed a shielding paradox: while classical planner-based arbitration intervened before the collision threshold at takeover moments, which was not verified for full runs, its conservative costmap constraints cause localized deadlocks in highly cluttered spaces. Future work will transition to multi-trial physical robot evaluations with dynamic obstacles, and investigate integrating formal safety bounds directly into reinforcement learning architectures to sustain safe, unshielded autonomy in narrow geometries.

Ethical statement. As the study did not involve human or animal subjects, formal ethical approval was not required.

Funding. This research received no specific grant from any funding agency in the public, commercial, or not-for-profit sectors.

Conflicts of interest. The author declares that there are no conflicts of interest regarding the publication of this paper.

Use of generative AI. Gemini, Claude, and ChatGPT were used in the editing and enhancement of the grammar and clarity of this manuscript. However, the ideas, arguments, conceptual framework, interpretations, and final content are entirely those of the author, who retains full responsibility for the accuracy, integrity, and final version of the paper.

Author Proof

References

Ames, A. D., Xu, X., Grizzle, J. W., & Tabuada, P. (2017). Control barrier function based quadratic programs for safety critical systems. *IEEE Transactions on Automatic Control, 62*(8), 3861–3876. https://doi.org/10.1109/TAC.2016.2638961

Broad, A., Murphey, T., & Argall, B. (2017). Learning models for shared control of human-machine systems with unknown dynamics. In *Proceedings of Robotics: Science and Systems XIII*. https://doi.org/10.15607/RSS.2017.XIII.056

Ciftci, Y. U., Chiu, D., Feng, Z., Sukhatme, G. S., & Bansal, S. (2025). SAFE-GIL: SAFEty guided imitation learning for robotic systems. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*.

Herbert, S. L., Chen, M., Han, S., Bansal, S., Fisac, J. F., & Tomlin, C. J. (2017). FaSTrack: A modular framework for fast and guaranteed safe motion planning. In *Proceedings of the 56th IEEE Conference on Decision and Control* (pp. 2242–2253).

Kelly, M., Sidrane, C., Driggs-Campbell, K., & Kochenderfer, M. J. (2019). HG-DAgger: Interactive imitation learning with human experts. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)* (pp. 8077–8083).

Koenig, N., & Howard, A. (2004). Design and use paradigms for Gazebo, an open-source multi-robot simulator. In *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)* (pp. 2149–2155).

Laskey, M., Lee, J., Fox, R., Dragan, A., & Goldberg, K. (2017). DART: Noise injection for robust imitation learning. In *Proceedings of the 1st Annual Conference on Robot Learning* (pp. 143–156).

Lee, G., Heo, J., Kim, D., Cheon, G., Oh, J., Hwang, M., Park, C., Lee, K., & Oh, S. (2026). SafeIL: Safety constrained imitation learning for autonomous systems. *Robotics and Autonomous Systems, 199*, Article 105376. https://doi.org/10.1016/j.robot.2026.105376

Macenski, S., Martín, F., White, R., & Clavero, J. G. (2020). The Marathon 2: A navigation system. In *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)* (pp. 2718–2725).

ONNX Runtime developers. (2021). *ONNX Runtime* [Computer software]. https://onnxruntime.ai/

Reddy, S., Dragan, A., & Levine, S. (2018). Shared autonomy via deep reinforcement learning. In In *Proceedings of Robotics: Science and Systems XIV*.

ROBOTIS. (n.d.). *TurtleBot3*. TurtleBot3 e-Manual. https://emanual.robotis.com/docs/en/platform/turtlebot3/overview/

Ross, S., Gordon, G. J., & Bagnell, J. A. (2011). A reduction of imitation learning and structured prediction to no-regret online learning. In *Proceedings of the Fourteenth International Conference on Artificial Intelligence and Statistics* (pp. 627–635).

Sha, L. (2001). Using simplicity to control complexity. *IEEE Software, 18*(4), 20–28.

Trautman, P., & Krause, A. (2010). Unfreezing the robot: Navigation in dense, interacting crowds. In *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems* (pp. 797–803). IEEE.

Wabersich, K. P., & Zeilinger, M. N. (2021). A predictive safety filter for learning-based control of constrained nonlinear dynamical systems. *Automatica, 129*, Article 109597. https://doi.org/10.1016/j.automatica.2021.109597

Williams, G., Drews, P., Goldfain, B., Rehg, J. M., & Theodorou, E. A. (2017). *Information-theoretic model predictive control: Theory and applications to autonomous driving*. arXiv. https://arxiv.org/abs/1707.02342

Zhang, J., & Cho, K. (2017). Query-efficient imitation learning for end-to-end simulated driving. In *Proceedings of the AAAI Conference on Artificial Intelligence* (Vol. 31, No. 1, pp. 2891–2897). https://doi.org/10.1609/aaai.v31i1.10857

Author Proof